SE4031 – Games Development

Assignment 01

“Galactic Defender” 2D Interactive Game

IT22100498

BANDARA M.R.J.K

1. Summary of the Game Concept

Galactic Defender is a 2D side-scrolling space shooter developed in Unity. The player controls a spaceship that must survive against waves of incoming enemy ships and asteroids while collecting health packs to stay alive. The game progressively increases in difficulty as the speed of obstacles rises over time, challenging the player to survive as long as possible and achieve the highest score.

Key Gameplay Loop

- **Move** the spaceship to dodge enemies and asteroids
- **Shoot** projectiles to destroy enemies and earn points
- **Collect** health packs to restore health
- **Survive** as long as possible as difficulty increases over time

The game features a main menu, in-game pause functionality, and a game over screen with restart and exit options. The player's score is tracked and displayed on the UI throughout the gameplay session. The health system provides visual feedback through a health bar that depletes as the player takes damage.

2. Core Features Implementation

2.1 Health System

The health system is managed by the **HealthManager.cs** script attached to the player. The player starts with a maximum health of **100**, displayed as a fill bar on the UI using Unity's `Image.fillAmount` property. When the player collides with an enemy, asteroid, or obstacle wall, `TakeDamage()` is called, reducing health and clamping it between 0 and the max value. When health reaches 0, the Game Over screen is displayed via `GameOverManager`. Health resets to full when the game restarts via the `Start()` method.

Script: HealthManager.cs

2.2 Score System

The score is managed through a singleton **ScoreManager.cs** script. Score is displayed as a `TextMeshProUGUI` element on the UI. Players earn **+10 points** by passing through obstacle wall gaps (via `ScoreTrigger.cs`) and **+50 points** by destroying enemies with projectiles. The score resets to 0 on game restart through the `ResetScore()` method called in `Start()`.
Scripts: ScoreManager.cs, ScoreTrigger.cs

2.3 Health Packs

Health packs are collidable objects spawned by the **Spawner.cs** script with a **10% spawn chance**. When the player collides with a health pack, the HealthPack.cs script triggers Heal(20f) on the player's HealthManager, restoring 20 health (clamped to the max value). The health pack is destroyed immediately on pickup. If the player misses it, the health pack is automatically destroyed when it moves off-screen (x < -15f).

Scripts: HealthPack.cs, Spawner.cs

2.4 Projectile Attack

The player fires projectiles using the **left mouse button** or **spacebar**, handled by **PlayerController.cs**. A shooting sound effect plays each time the player fires. Projectiles are instantiated at the player's fire point position and travel to the right at a speed of 20 units/second. Each projectile is automatically destroyed after **2 seconds** (lifetime) or upon collision with an enemy or wall. The **Projectile.cs** script handles collision detection, scoring, blast sound effects, and cleanup.

Scripts: PlayerController.cs, Projectile.cs

2.5 Enemy Mechanics

Enemies are spawned from the right side of the screen by **Spawner.cs** with a **60% spawn chance**. They move leftward at a base speed multiplied by the global difficulty speed. Enemies are destroyed on collision with the player (dealing 20 damage with a blast sound) or when hit by a projectile (awarding 50 score points with a blast sound). Enemies that pass off-screen (`x < -15f`) are automatically cleaned up. Additionally, **asteroids** spawn with a **30% chance**, featuring random rotation and varied speeds for visual variety, and also trigger blast sounds on collision.

Scripts: EnemyController.cs, Astroid.cs, Rotator.cs, Spawner.cs

2.6 Obstacle Wall Corridors

Obstacle wall corridors are a key gameplay mechanic that tests the player's navigation skills. Walls are spawned on a **separate timer** from regular enemies, with a starting interval of **8 seconds** that decreases to **5 seconds** as difficulty increases. Multiple corridor prefab variants are supported for variety — the spawner randomly selects from an array of prefabs.

Each wall corridor contains solid wall segments with gaps that the player must fly through. The **ObstacleWall.cs** script handles wall segments — when the player collides with a wall, they take **20 damage** with a blast sound effect, but the wall is **not destroyed** (the player passes through it). A **damage cooldown** of 1 second prevents rapid damage stacking.
The **ObstacleController.cs** script handles the overall corridor movement, moving leftward with speed scaling based on the difficulty multiplier. Score triggers placed in the gaps award **+10 points** when the player successfully navigates through.

2.7 Speed Increment

The **DifficultyManager.cs** script manages a static `globalSpeed` variable that starts at **1.0** and increases by **0.05 per second**. This multiplier is applied to enemy movement speed and spawn rate, making the game progressively harder. The speed is capped at a maximum of **3.0** to ensure the game remains playable. The speed multiplier resets to 1.0 when the game restarts.

Script: DifficultyManager.cs

2.8 Sound & Audio System

The game features a comprehensive audio system to enhance the gameplay experience:

- *Background Music: Looping background track played via an AudioSource on the BackgroundMusic GameObject*
- *Shoot Sound: Laser sound effect triggered via AudioSource.PlayOneShot() each time the player fires (PlayerController.cs)*
- *Blast Sound: Explosion sound played via AudioSource.PlayClipAtPoint() when enemies or asteroids are destroyed (Projectile.cs, EnemyController.cs, Astroid.cs)*
- *Health Pickup Sound: Pickup chime played via AudioSource.PlayClipAtPoint() when collecting a health pack (HealthPack.cs)*

PlayClipAtPoint() is used for objects that are destroyed on impact — it creates a temporary AudioSource at the collision position that persists after the object is destroyed, ensuring the sound plays fully.

3. Control Guide

Action	Input	Description
Move Up	W / ↑	Move the spaceship upward
Move Down	S / ↓	Move the spaceship downward
Move Left	A / ←	Move the spaceship to the left
Move Right	D / →	Move the spaceship to the right
Shoot	Left Mouse Button / Space	Fire a projectile to the right
Pause	Escape	Pause / Unpause the game

Menu Controls

Screen	Buttons Available
Main Menu	Play, Quit
Pause Menu	Resume, Main Menu
Game Over Screen	Restart, Exit

4. Creative Feature – Screen Shake

The creative feature implemented in this game is a dynamic screen shake effect that provides impactful visual feedback during gameplay collisions. The feature is managed through the CameraShake.cs singleton script attached to the main camera.

How It Works

The screen shake system uses a coroutine-based approach that temporarily offsets the camera's position from its original coordinates using randomized values within a configurable magnitude range. The effect has two parameters:

- Duration: How long the shake effect lasts (in seconds)
- Magnitude: How intense the shake is (the displacement range)

When It Triggers

Event	Duration	Magnitude	Feel
Player hit by enemy/asteroid	0.2s	0.3	Heavy, impactful shake
Enemy destroyed by projectile	0.1s	0.1	Light, satisfying shake

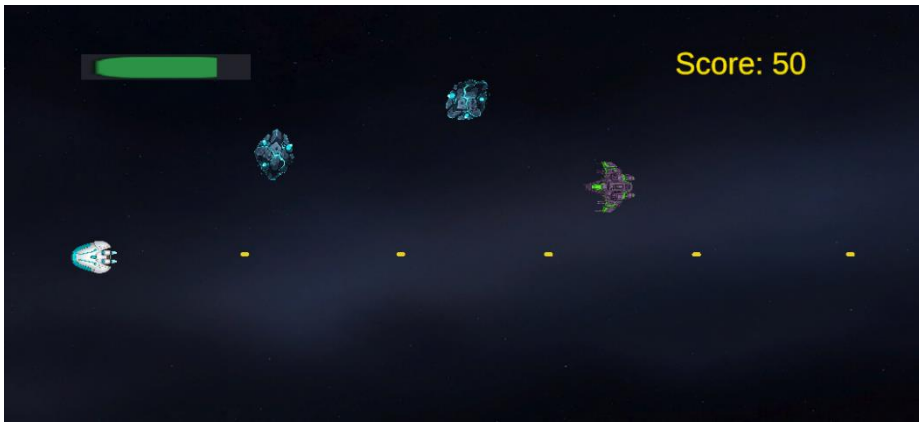
Technical Implementation

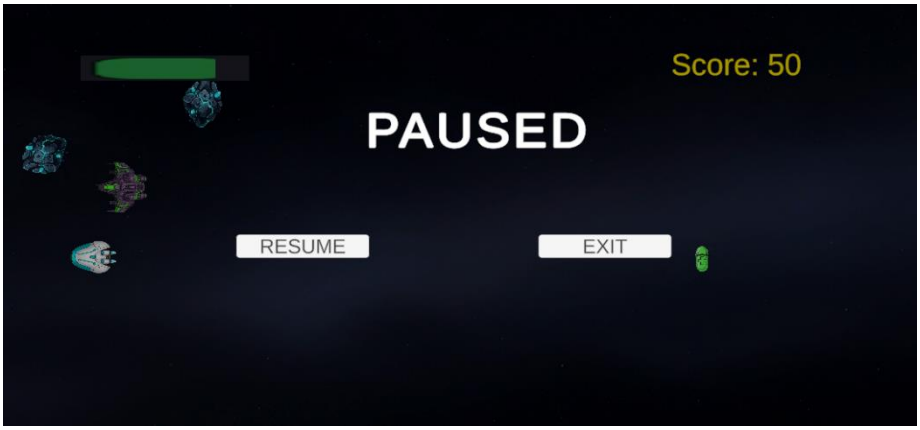
The ShakeCoroutine runs a loop for the specified duration. Each frame, it generates random X and Y offsets within the magnitude range and applies them to the camera's position. Once the duration elapses, the camera snaps back to its original position.
A StopShake() method was implemented to immediately halt any active shake and reset the camera. this is called when the Game Over screen appears to prevent the shake effect from persisting on the frozen screen.

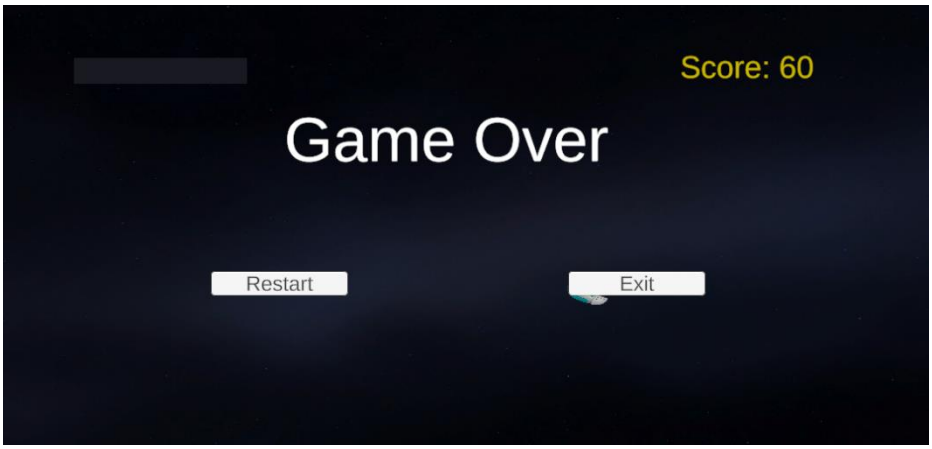
Impact on Gameplay

The screen shake adds a visceral, physical quality to the game. The differentiation between heavy and light shakes helps the player distinguish between taking damage (bad) and destroying enemies (good), improving the overall game feel and player engagement without adding UI clutter.

5. Screenshots







6. Credits & External Assests

Game Engine

Tool	Source	Usage
Unity Game Engine	unity.com	Game development platform
TextMeshPro	Unity Technologies (built-in)	UI text rendering

Visual Assets — Generated with Google Whisk

Asset	Tool	Usage
Player Spaceship Sprite	Google Whisk (AI Image Generation)	Main player character prefab
Enemy Ship Sprite	Google Whisk (AI Image Generation)	Enemy prefab visuals
Asteroid Sprite	Google Whisk (AI Image Generation)	Asteroid obstacle prefab
Health Pack Icon	Google Whisk (AI Image Generation)	Health pack collectible prefab
Projectile Sprite	Google Whisk (AI Image Generation)	Bullet/laser projectile prefab
Background Wallpapers	Google Whisk (AI Image Generation)	Space-themed game backgrounds
UI Icons & Elements	Google Whisk (AI Image Generation)	Menu buttons, health bar, and UI decorations

Audio Assets — Generated with Suno AI

Asset	Tool	Usage
Background Music	Suno AI (AI Music Generation)	Looping background track during gameplay
Laser Shoot Sound	Suno AI (AI Audio Generation)	Sound effect when player fires a projectile
Blast/Explosion Sound	Suno AI (AI Audio Generation)	Sound effect on enemy/asteroid destruction
Health Pickup Sound	Suno AI (AI Audio Generation)	Sound effect when collecting a health pack

Note: All visual assets (sprites, backgrounds, icons) were generated using **Google's Whisk** (labs.google/whisk), an AI-powered image generation tool. All audio assets (music, sound effects) were generated using **Suno AI** (suno.com), an AI music and audio generation platform. No third-party copyrighted assets were used.

7. Gameplay Demo Video

https://youtu.be/CB91IDVa5fs

Appendix: Project Scripts

Script	Purpose
`PlayerController.cs`	Player movement, shooting input, shoot sound effect
`HealthManager.cs`	Health system, damage, healing, triggers game over on death
`ScoreManager.cs`	Score tracking and UI updates (singleton)
`ScoreTrigger.cs`	Score increment when passing through wall gaps
`Projectile.cs`	Bullet movement, collision, blast sound, scoring, cleanup
`EnemyController.cs`	Enemy movement, player collision, blast sound
`Astroid.cs`	Asteroid movement, rotation, collision, blast sound
`Rotator.cs`	Random spinning for visual variety
`HealthPack.cs`	Health pickup on collision, pickup sound
`ObstacleWall.cs`	Wall segment damage with cooldown, blast sound, non-destructible
`ObstacleController.cs`	Corridor movement with difficulty scaling, damage, blast sound
`Spawner.cs`	Enemy, asteroid, health pack, and corridor spawning with spawn rates
`DifficultyManager.cs`	Progressive speed increase over time (capped at 3x)
`CameraShake.cs`	Screen shake effect with stop functionality (creative feature)
`PauseManager.cs`	Pause/resume game, main menu navigation
`GameOverManager.cs`	Game Over screen, restart, exit, stops camera shake
`MainMenu.cs`	Main menu Play and Quit buttons